

Notes Week 29

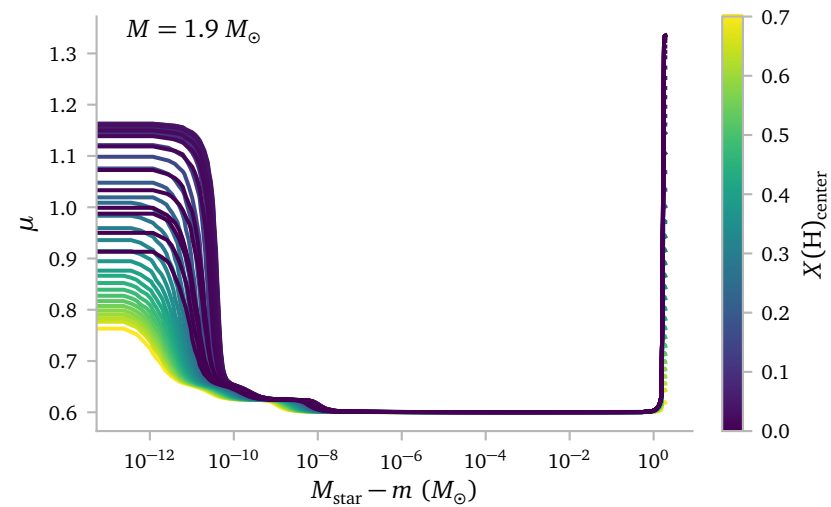
a. Thermohaline Mixing

I simulate a bunch of MS stars with different initial masses. I save profiles depending on the central hydrogen fraction to get an even coverage of the μ profiles throughout the MS evolution of the stars. This is important because the μ profiles change quite a lot throughout the MS evolution. We obviously know the age of the MS star when it accretes mass and thus we can make a pretty good assumption about the mixing.

The problem is then that we could possibly observe the MS stars billions of years after the mass has been accreted. At this point the μ profiles could change and it becomes uncertain.

However, since the abundances are quite uncertain anyway, I think it is okay to just take the μ profile at the time of accretion.

To start, let me plot the μ profile from the envelope to the center.

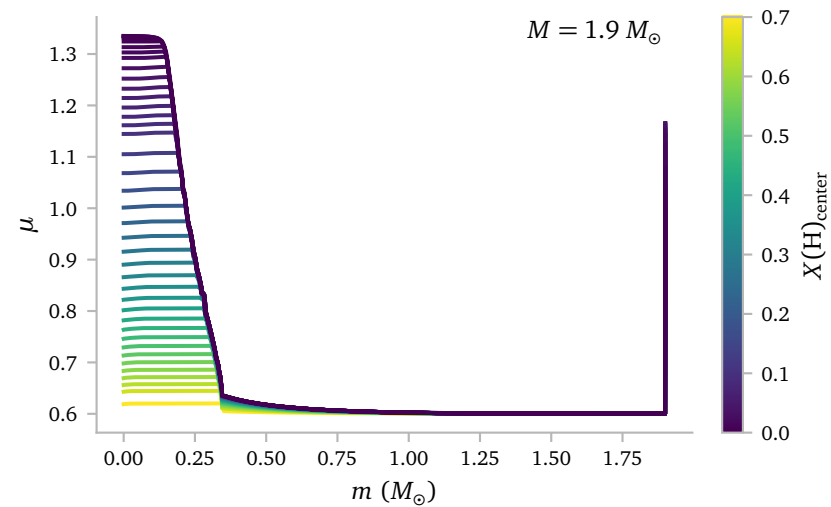


Here, we quite clearly see that outer layers of the envelope, which have very low density, are not fully ionized. This obviously causes the mean molecular weight to increase.

However, I think this is an unimportant effect. Accreted material will just the envelope layer further outwards, and the previously unionized material will become ionized.

The much more interesting part is close to the core.

Let's look at the profile going from the core outwards.

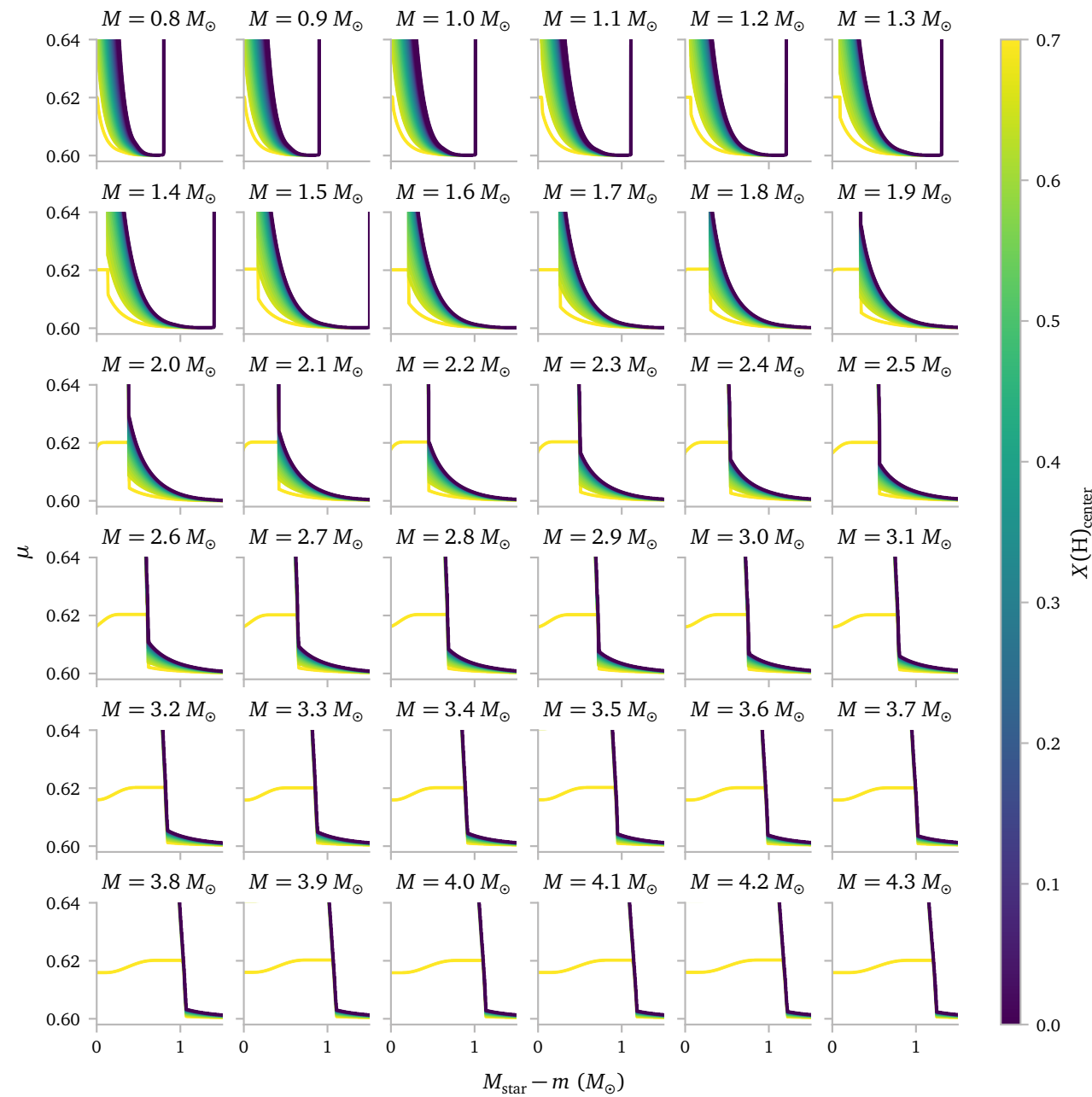


For this particular mass, we see a nearly perfectly diagonal line from the mean molecular weight in the core to the mean mean molecular weight in the envelope.

The diagonal line does not shift with the changing core composition, only the final height changes.

However, just before the “diagonal” line starts, the μ profile already starts to rise a bit. I think this is pretty interesting, as this seems to change quite a bit depending on the core composition, and could thus be important for the moment of accretion.

Below I plot all μ profiles for all models, zooming in on this lower portion.



Clearly, not only the mass position of the rise in μ changes with mass, but also the slow rise before the diagonal line.

However, I have no intuition for the mean molecular weight of some strongly enriched material, so before I do a bunch of calculations, let me make a really simple estimate for μ .

a.1 Simple Derivation

NOTE: Simple derivation for myself to refresh how the mean molecular weight is calculated.

The mean molecular weight is defined as:

$$\mu = \frac{\rho}{nm_u}.$$

Here,

- μ is the mean molecular weight of a gas.
- ρ is the mass density of a gas.
- n is the number density of a gas.
- m_u is the atomic mass unit.

For one element, the mass density contributed is

$$\rho_i = X_i \rho.$$

The number density of nuclei is then simply

$$n_i^{\text{nuc}} = \frac{\rho_i}{A_i m_u}.$$

However, if we assume full ionization, we will have 1 nucleus + Z_i electrons.

The “real” number density is simply

$$n_i = (1 + Z_i) n_i^{\text{nuc}}.$$

Now, let me expand and sum over all elements:

$$\begin{aligned} n &= \sum_i (1 + Z_i) \frac{\rho X_i}{A_i m_u} \\ &= \frac{\rho}{m_u} \sum_i \frac{X_i (1 + Z_i)}{A_i} \\ \frac{nm_u}{\rho} &= \sum_i \frac{X_i (1 + Z_i)}{A_i} \\ \frac{1}{\mu} &= \sum_i \frac{X_i (1 + Z_i)}{A_i} \end{aligned}$$

Cool! I am *assuming* full ionization here, which I think is fine. I am pretty sure nearly the whole MS star is ionized anyway.

Now to compute the mean molecular weight of the accreted gas, we need

- X_i , the mass fraction of every element in the accreted gas. We have this.
- Z_i , the proton number of every accreted element. We also have this.
- A_i , the mass number of every element in the accreted gas. We need to make an *assumption* here.

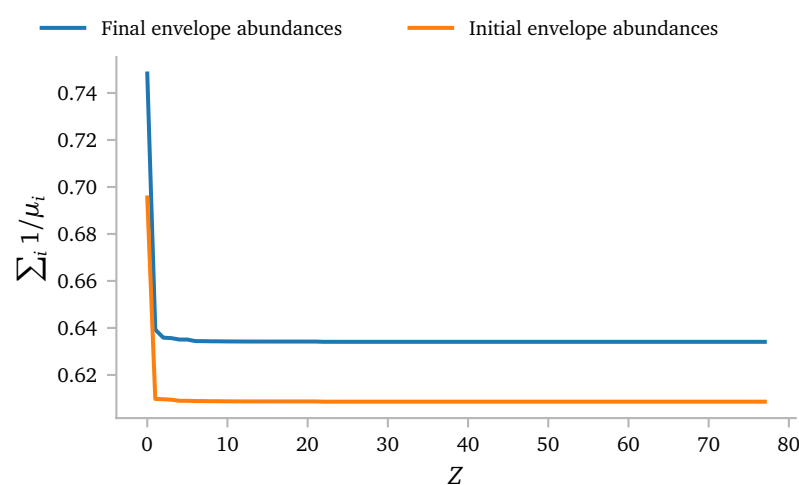
Especially for heavy elements, it is quite possible that the element has a few isotopes with different masses that are all stable and relatively abundant. In this case, the elemental mass should be the averaged mass of the isotopes.

I think it is fine to take the [standard atomic weight](#) in this case. The differences will be small anyway, especially for the heavy elements.

I will use the package `periodictable` written by [Kienzie et al. \(2026\)](#) based on data from [Meija et al. \(2016\)](#).

It is then quite straightforward to μ .

Below I plot the μ I calculated from my MESA model with the Monash Abundances.



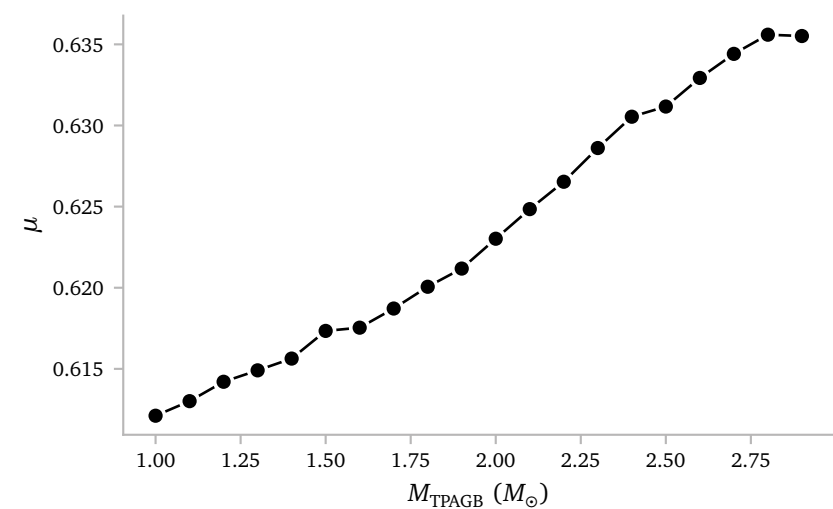
There are a couple of important things here.

1. The first elements contribute by far the most to μ , which is to be expected due to their much larger abundances.
2. The μ computed with the final abundances is larger than for the initial envelope abundances.
3. μ computed from the initial envelope abundances is a bit larger than μ in the envelope of the MESA MS models, which have $\mu \approx 0.60$.

I am unsure if the differences in μ between the initial models is due to evolution after the MS, or due to differences between MESA and Monash abundances. But, in the end, the differences for the predicted abundances in the MS star would probably be small.

I have also implemented this computation in the `Abundance` class that handles all abundance post-processing for me.

Below I show the averaged μ of all expelled material of the single star TPAGB models.



We clearly see that the average mean molecular weight of the expelled material rises with mass. I think this makes sense, as we also see more dredge-up at higher masses, and thus on average, more heavy elements in the envelope of these stars.

The idea is now to use this μ and mix the accreted mass, thereby changing μ as well, until the mixed μ is equal to the μ in the star.

a.2 Simple Derivation 2

To see how this mixing would work, let me look at the definition of μ again.

$$\mu = \frac{\rho}{nm_u} = \frac{m/V}{Nm_u/V} = \frac{m}{Nm_u}.$$

Say we mix m_1 of gas with mean molecular weight μ_1 with m_2 of a gas with μ_2 . We want to know what the resulting μ is of the mixed whole.

We would calculate N using μ and m to obtain:

$$N_i = \frac{m_i}{\mu_i m_u} \implies \mu_{\text{mix}} = \frac{m_1 + m_2}{(N_1 + N_2)m_u} = \frac{m_1 + m_2}{\frac{m_1}{\mu_1} + \frac{m_2}{\mu_2}}$$

Or, more conveniently:

$$\mu_{\text{mix}}^{-1} = \frac{m_1 \mu_1^{-1} + m_2 \mu_2^{-1}}{m_1 + m_2}, \quad m_{\text{mix}} = m_1 + m_2.$$

I think it should be pretty easy to implement this logic for simulating the dilution of μ as the accreted material starts to mix.

Here is a minimal working example:

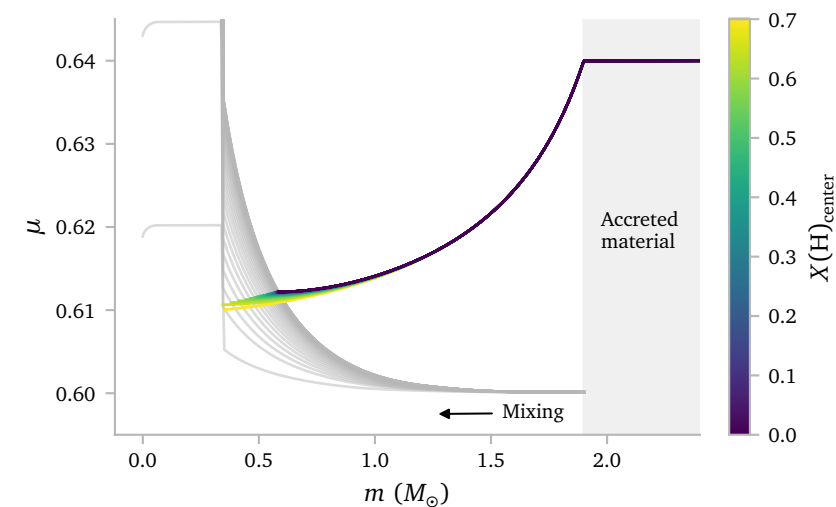
```
def effective_mu(profile, M_acc, mu_acc):
    ms = -np.diff(profile.mass)
    mus = (profile.mu[1:] + profile.mu[:-1]) / 2
    arg = np.argmin(mus)
    mus[arg] = mus[arg]
    mu_current = mu_acc
    m_current = M_acc

    mu_profile = [mu_acc]

    crossing_index = None
    for m, mu in zip(ms, mus):
        mu_last = mu_current
        mu_current = (m + m_current) / ((m / mu) + (m_current / mu_current))
        m_current = m + m_current
        mu_profile.append(mu_current)

    mus = np.concatenate([[mus[0]], mus])
    mu_profile_original = mus
    return mu_profile, mu_profile_original
```

Below I show an example of $0.5 M_\odot$ being accreted by a $1.9 M_\odot$ MS star at various times throughout the MS.

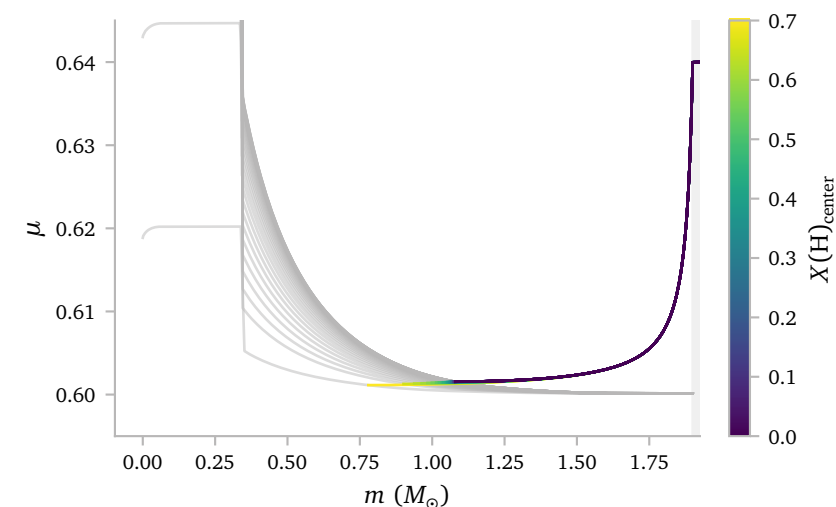


Here, accreted mass gets added to the top of the envelope of the original star.

As mixing extends into the envelope, the mean molecular weight of the mixed material decreases from the initial accreted mean molecular weight to lower values until at some point, the mean molecular weight of the mixed material is *lower* than the mean molecular weight of the envelope of the original star.

This critical point roughly determines how deep mixing extends.

To me, this seems like it works as expected. Another test would be to see how mixing works with much less accretion. In this case, mixing should cause the mean molecular weight to drop much quicker.



This plot show the same $1.9 M_\odot$ model accreting “only” $0.025 M_\odot$ of strongly enriched mass.

And indeed, we see a much quicker drop-off and *less* mixing.

We also see that the age of the MS star does change the mixing depth a bit. In both case I have shown here, the mixing depth decreases with age.

This is a nice property, because by using the μ profile at the time of accretion, we are making an *optimistic* estimate for the amount of mixing, and thus a *conservative* estimate for the abundances.

Now we are quite easily able to obtain the mixing mass. In combination with the amount of accreted mass and the mass fraction of the elements in the accreted mass, we are almost ready to compute the envelope abundances in the main sequence stars.

a.3 Interpolated mean molecular weight calculations.

The basic idea is very similar to the interpolation of the abundances. First, I interpolate the μ profiles for different ages¹. Then I interpolate these profiles between the masses.

a.3.1 Main sequence age to central Hydrogen composition.

This is just a simple linear interpolation as well.

Below I print the implementation. The only important thing here is that it is interpolated over the mass as well. This means I get a central hydrogen composition based on the mass and age of the main sequence star.

We only need the mass and the age if we assume mass-transfer happens much quicker than the evolution of the MS star². We simply use the mass of the main sequence star before mass transfer, so $q \times M_{\text{TPAGB},i}$ and the age of the TPAGB star at the moment of RLOF. We implicitly assume that the stars are born at the same time³.

```
class Accretor:
    ...

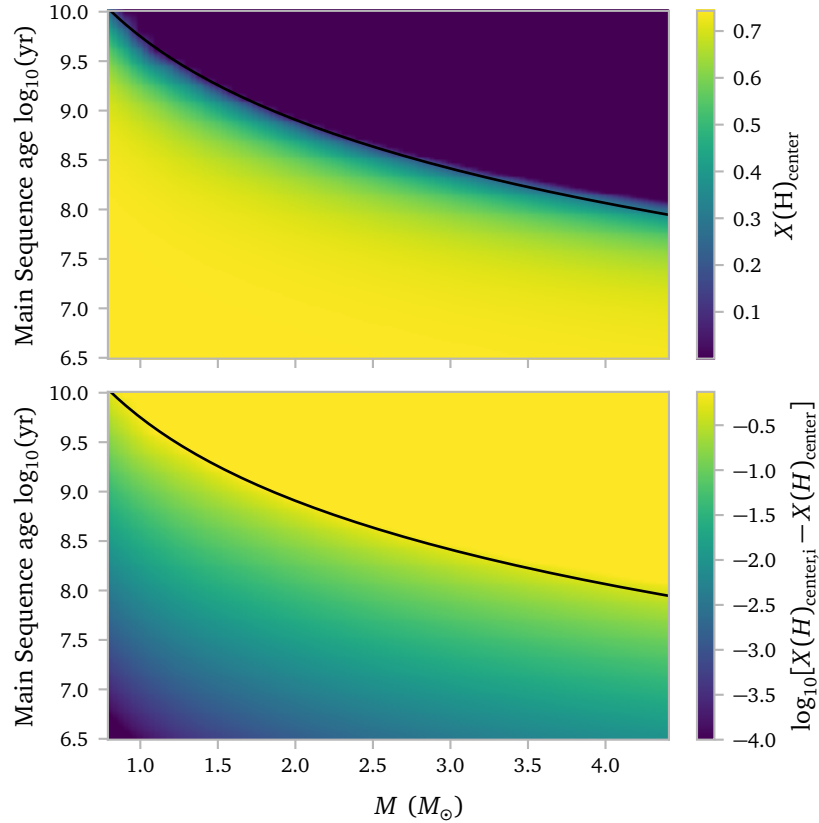
    def __get_center_h1(self) -> float:
        interp_ages: list[float] = []
        for mass in self.masses:
            mass_profiles: list[AccretorProfile] = self.profiles[mass]
            h1s: list[float] = []
            ages: list[float] = []
            for profile in mass_profiles:
                h1s.append(profile.center_h1)
                ages.append(profile.age)

            interp_ages.append(np.interp(self.age, ages, h1s))

        return np.interp(self.mass, self.masses, interp_ages)

    ...
```

This is the result of the interpolation:



I think it works well. There are some blocky artifacts for the lower masses, but I do not expect this error to significantly propagate through to the abundances.

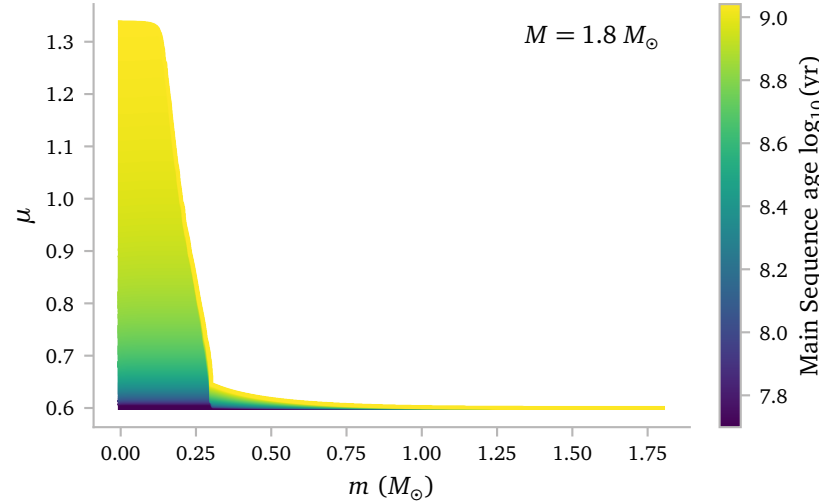
We can also clearly see that the Main Sequence lifetime decreases with initial mass. I have plotted the simple relation $\tau_{\text{MS}} \approx 10^{9.75} \cdot M^{-2.8}$, where I manually fit the factor $10^{9.75}$ to the interpolation results.

The bottom plot shows that the interpolation also works quite well for early ages, even though we have relatively few profiles in this region.

a.3.2 Interpolated Age and Mass

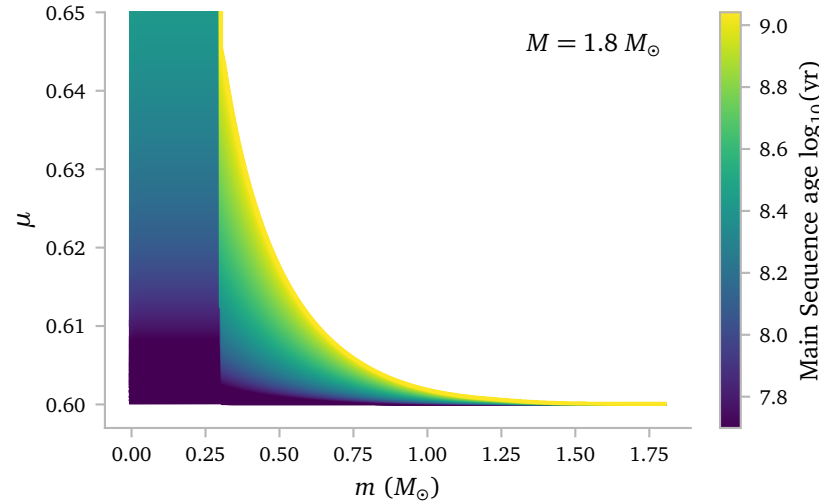
Now that we have the mass and central hydrogen composition at the moment of mass transfer, we can use these values to generate an interpolated mass-mean molecular weight profile. The idea is quite simple. We interpolate 4 profiles of the surrounding masses and central hydrogen compositions.

I have created a bit of code⁴ that can quickly generate interpolated profiles. Below, I show two examples. In the first example, I change the age of a MS star to see how the μ profile changes.



We can clearly see that, as the star becomes older, the mean molecular weight of the star in the core rises. This is to be expected as hydrogen turns into helium.

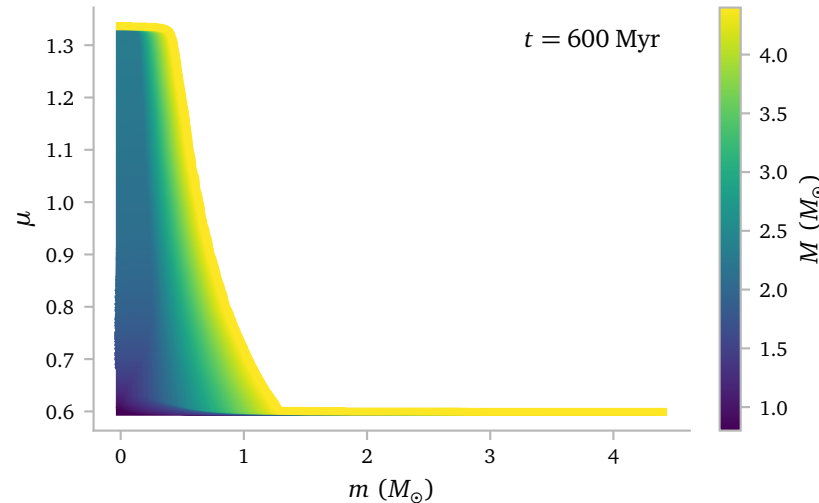
We can also see that the slow sharp rise happens at the same masses for all ages. However, the slow rise before the sharp rise to the core mean molecular weight value does depend on age. In the next figure, I zoom in on this.



The zoom very clearly shows the sharp boundary between the slow and fast rise of the mean molecular weight *and* the effect of age.

This is really the regime that we are interested in, because the mean molecular weight of the accreted mass will in all cases be pretty close to my estimated value of 0.64.

We can also look different masses at a constant age.

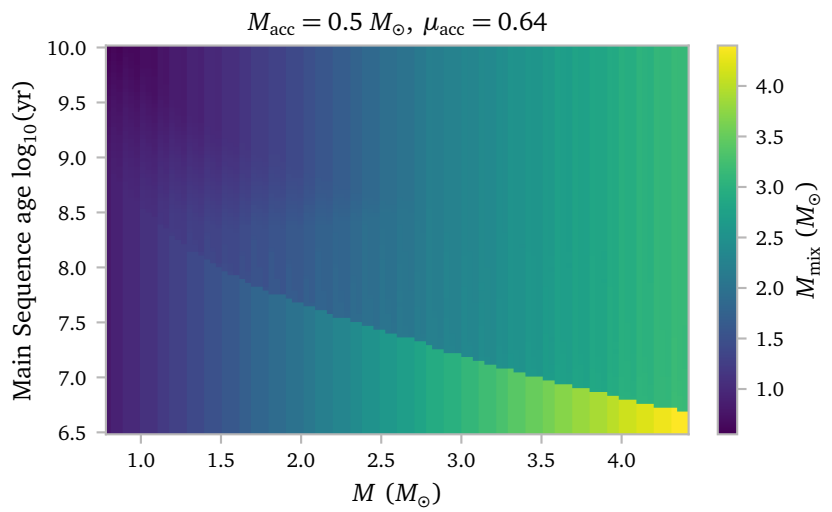


Now, we can very clearly see that the mass position of the rise in molecular weight changes with mass. This makes perfect sense. It is really just a proxy for the core mass of the star anyway, and lighter stars will have lighter cores.

Since we are comparing stars of different masses at the same age, we also see that the maximum μ in the core increases with mass too. This also makes perfect sense, since heavier stars have shorter main-sequence lifetimes, and thus, at the same age will have burned through more of their core hydrogen.

a.3.3 Inspect Mixing

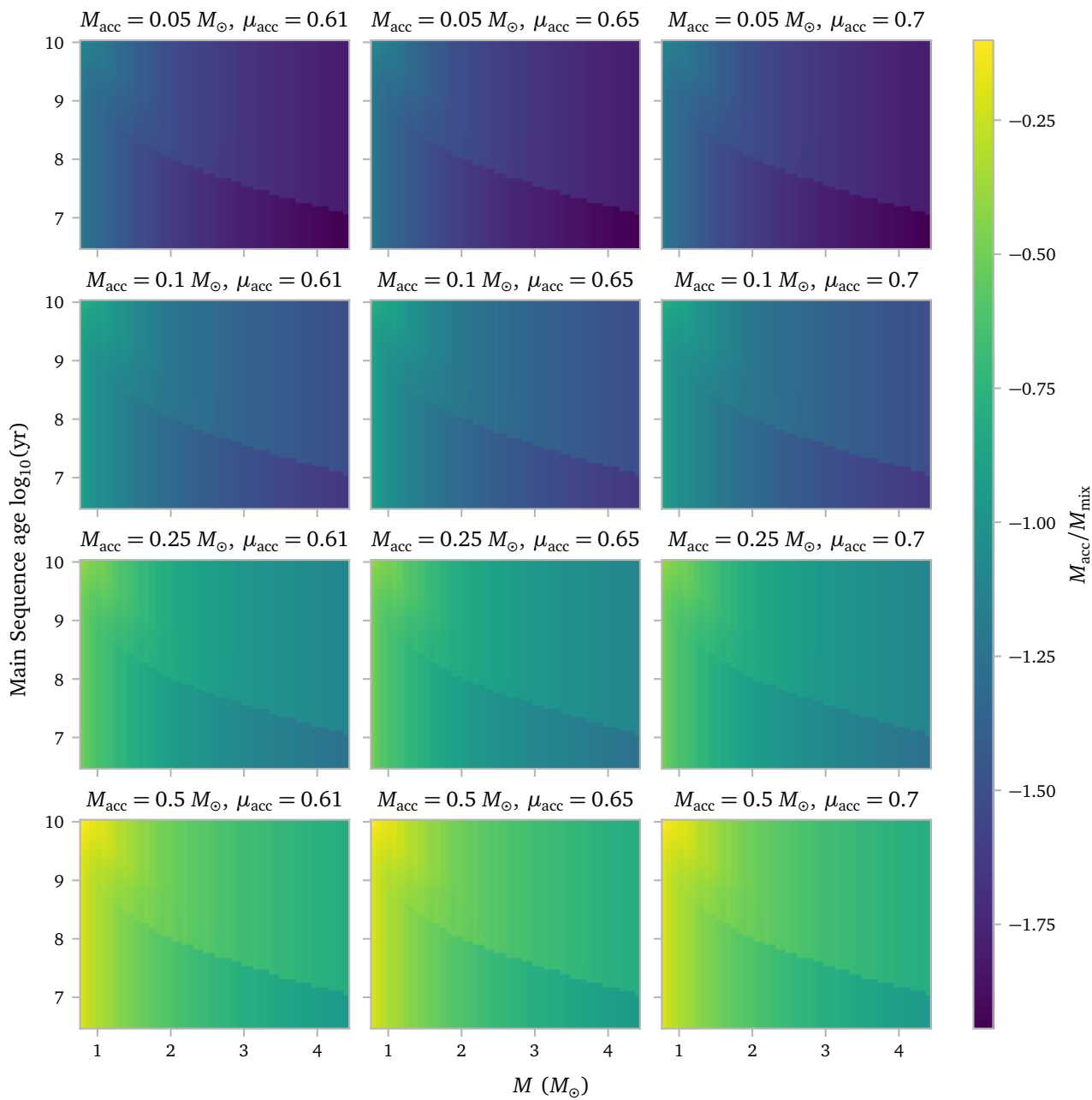
If we hook this interpolation up to the mixing propagation calculation, we can compute the mixing depth for any profile in combination with any accreted material. Below I show an example:



Here I show with how much MS envelope mass the accreted material is mixed before the mean molecular weight is equal to the increased mean molecular weight towards the core.

- There are clearly two regimes.
- The regime where the core is not enriched enough yet, and we mix throughout the entire star.
 - The regime where the core has burned enough hydrogen, and we stop mixing before or at the core boundary.

The final abundances of the MS star obviously depend both on the amount of accreted mass *and* the amount of mixing. To show this effect, we look at a grid of different μ_{acc} and M_{acc} and compare $M_{\text{acc}}/M_{\text{mix}}$.



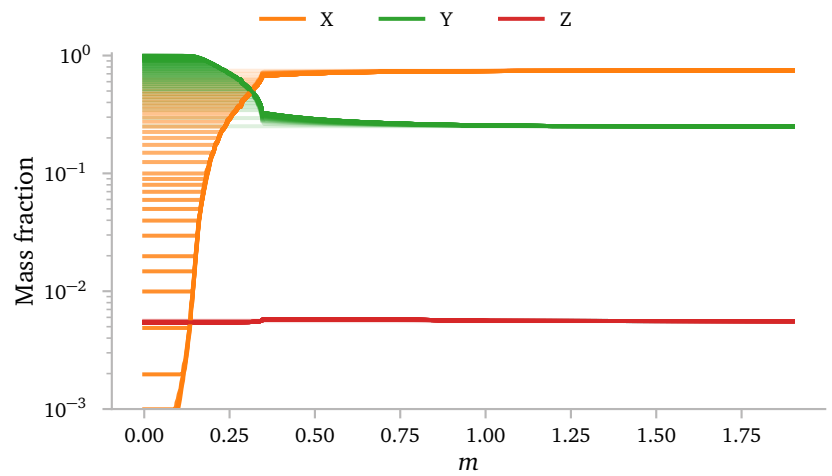
It is very clear that the amount of accreted mass is very important for the final abundances. In one of the plots above, we saw that accreting a smaller amount of mass means we mix less as well, but this effect does not in any way compensate the fact that more accretion simply means there are more heavy elements in the MS star.

This makes intuitive sense too. When accreting more enriched material, we expect larger final abundances. We also see that the actual mean molecular weight of the accreted material makes *very little* difference for the ratio between the accreted mass and the mixed mass.

a.4 Main Sequence Composition

To compute the abundances after accretion, we need to know the abundances of the accreted material. We also need to know how much this material is mixed with the main sequence star. Lastly, we need to know the abundances of the mixing material.

However, as is clear from the μ -profiles, the abundances are not constant in the envelope of the main sequence star. Let me investigate if this is only Helium leaking out from the core. In that case, I am not too worried about the abundances of heavier elements changing much inside the envelope and with time.



Compared to the changes in the hydrogen and helium mass fraction, the metal mass fraction hardly changes with time and internal mass.

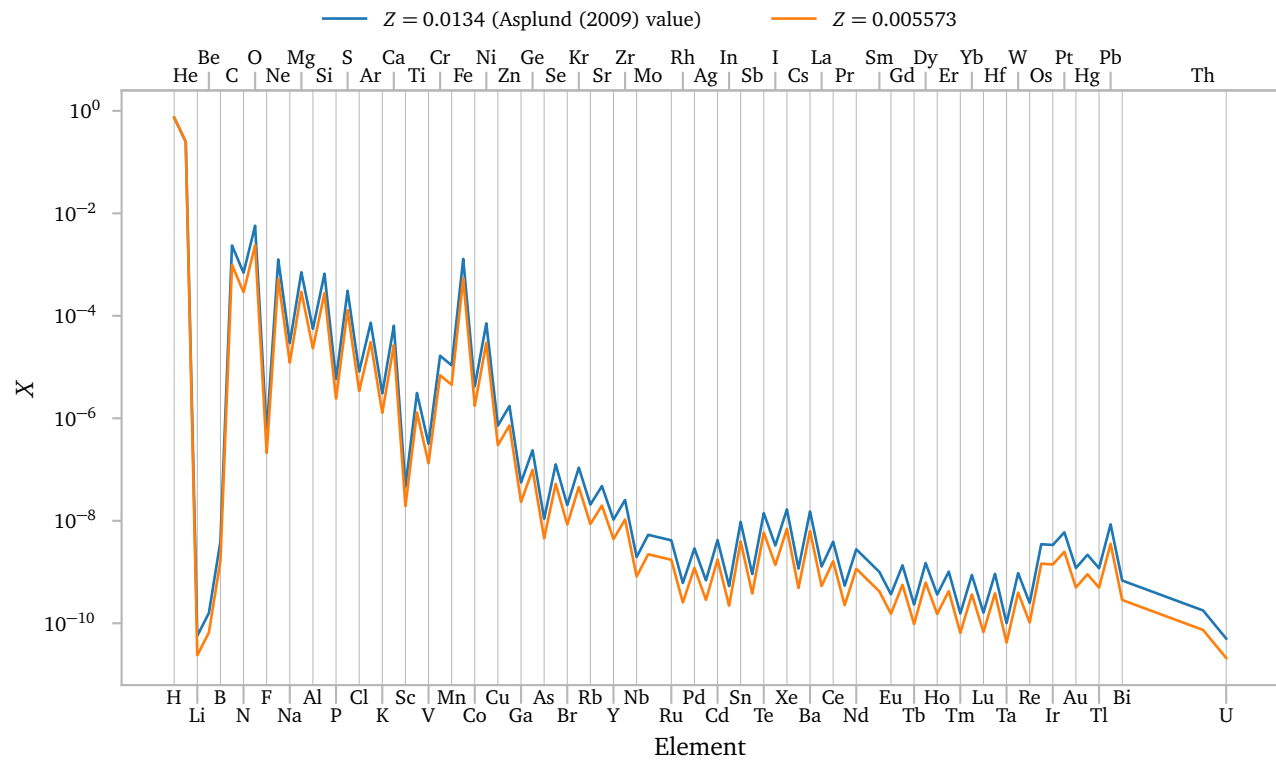
Although this is not a strict proof that the abundances of the individual elements will not change much either, I think it is good evidence.

I therefore feel comfortable to just use the surface abundances from Asplund et al. (2009), which we use in MESA as well.

I used the elemental abundances table from Asplund et al. (2009) for the elemental abundances at the surface of the sun. I then scale the abundances to a different metallicity.

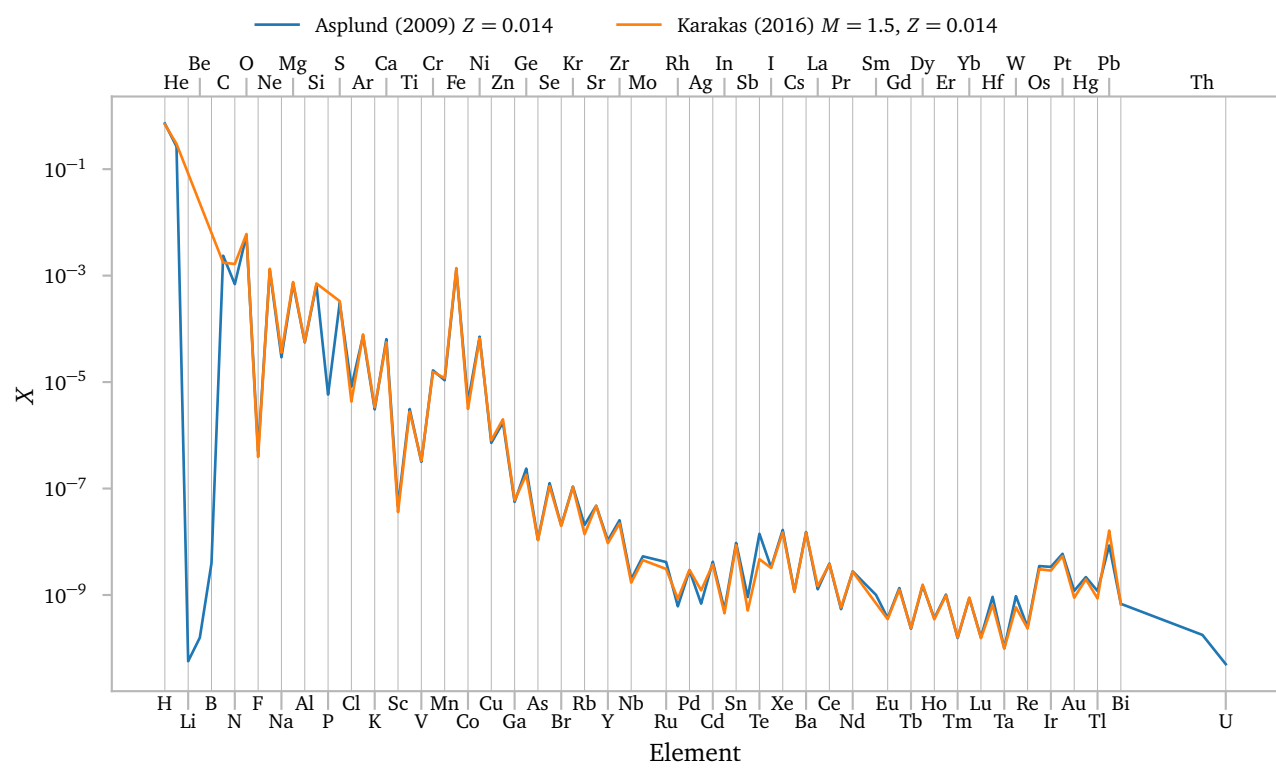
For this, I use the convention that $Y = 0.24 + 2Z$ and $X = 1 - Y - Z$, which my MESA models all use as well. This means we *know* the mass fraction of helium in our stars. We also *know* the mass fraction of hydrogen in our stars. We also know Z in our stars, and we can compute Z in the Asplund et al. (2009) abundances. To obtain the scaled elemental abundances of the other elements, I simply scale these by the metallicity.

Below, I plot the original mass fractions and the scaled mass fractions.



Looks like it works to me.

Now, let me compare with the initial envelope abundances of the Monash models.



The two seem to agree very well, which makes sense, as Karakas & Lugaro (2016) explicitly mention that they use Asplund et al. (2009) abundances.

This also means that⁵, the envelope abundances really do not change between the MS and the TPAGB.

I think the easiest way to proceed is to simply reuse the initial envelope abundances when computing dilution due to mixing.

⁵ At least under the assumptions from Karakas & Lugaro (2016)

b. Optimizations

b.1 Numpy matrices

Since I now need to compute every element for every binary model to compute μ and properly infer the amount of mixing, it became important to optimize my abundance post-processor.

Before, I interpolated every isotope individually. However, if I allow numpy to work on arrays of abundances, I can interpolate every isotope at the same time, which should in theory be quicker.

I have implemented this, and it seems to be around 3 to 4 times quicker this way without any loss in quality. I also still have access to the methods to obtain the abundances for a single element if this may be required.

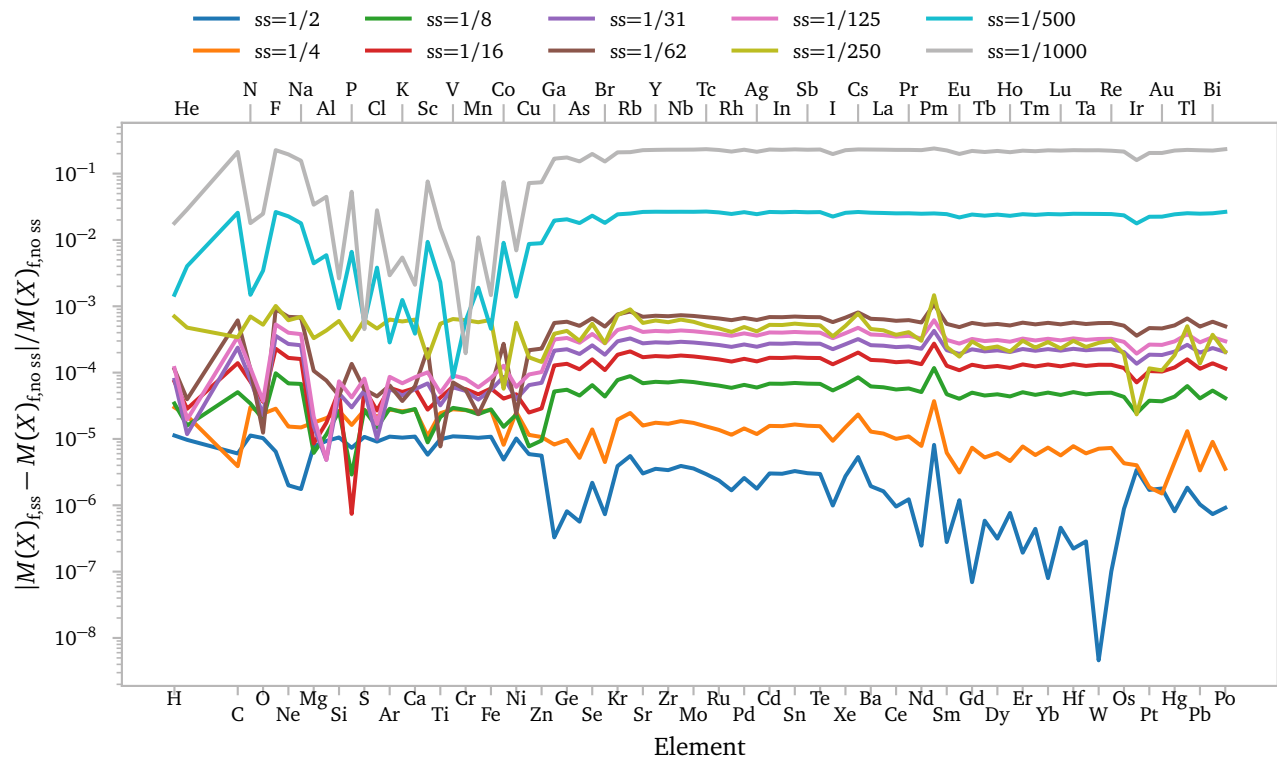
Although this does not give us any interesting results now, it will definitely help me later when I will need to compute all abundances for a bunch of stars.

b.2 Downsampling

Another really easy optimization is to downsample the input models. Since I am using the MESA models with history data saved at every timestep for accurate and consistent orbital evolution, the models contain many thousands of steps⁶. This level of resolution is absolute overkill for the abundance computations, and I think we can get away with downsampling by a factor 10 to 100.

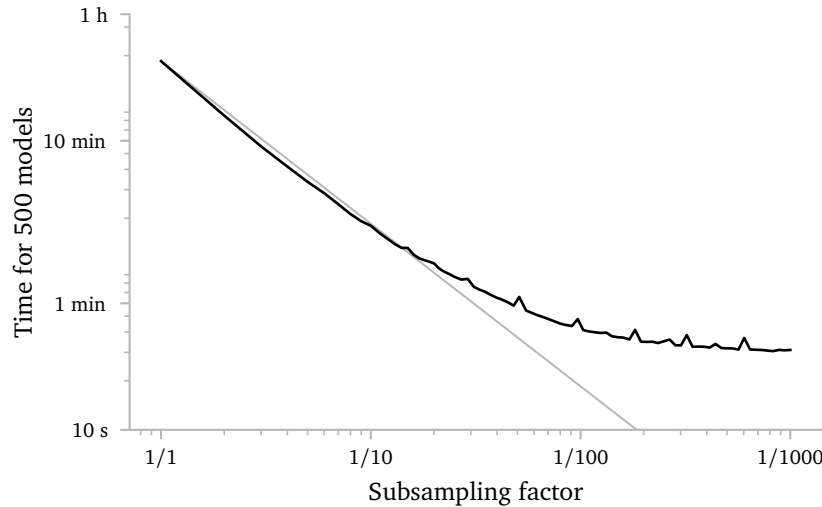
To show the effect of downsampling, let me show the effect on the final yields. In the figure below, I plot several final yields with different subsampling settings.

⁶ Sometimes more than 100k



I think it is quite clear that, even for quite large subsampling values of 1/100, the resulting accuracy is more than enough to not matter in the end.

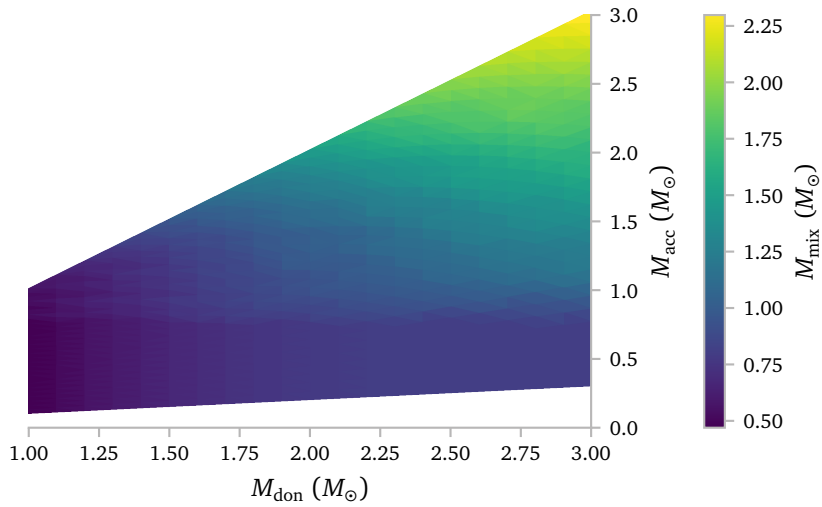
The speed-up of the subsampling however is important for quickly analysing results:



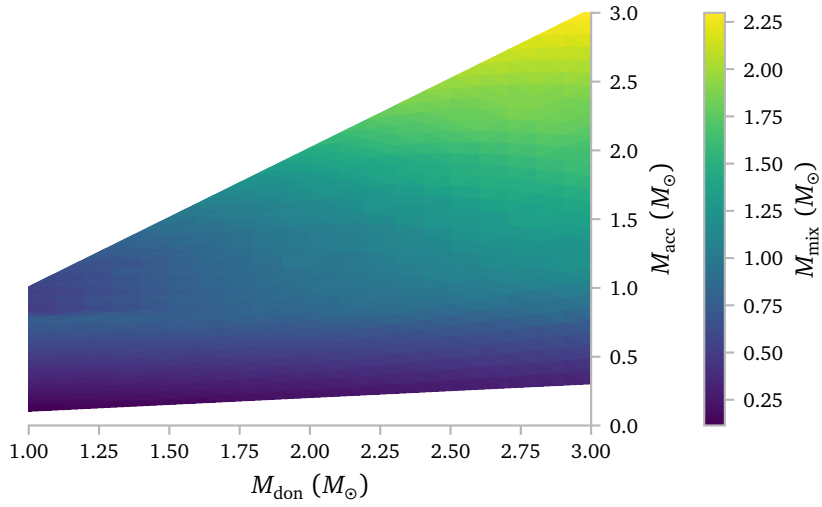
Clearly, initially, subsampling by a factor x directly leads to a speed-up by this same factor x . At some point, this relation no longer holds as we approach the regime where the length of the arrays is no longer the bottleneck in the computation time.

This also shows that, by taking a subsampling factor of 1/100, we can analyse quickly analyse even very large MESA grids.

c. Combining everything



I think the figure makes sense. However, we should be careful to interpret the lower masses. Here, I do not have main sequence models because MESA was not able to simulate these stars using the ÆSOPUS table. Therefore, I think we should just use the assumption that the mass is mixed throughout the whole star in this case.



This might not work so well for TPAGB masses below 1.5 $M_{\odot}$, but I think these will not be so important for the barium star production anyway.

References

Asplund, M., Grevesse, N., Sauval, A. J., & Scott, P. (2009) *The Chemical Composition of the Sun*. *ARA&A***47**, 481.

Karakas, A. I. & Lugaro, M. (2016) *Stellar Yields from Metal-rich Asymptotic Giant Branch Models*. *ApJ***825**, 26.

Kienzle, P., Maranville, B. B., Cooper, J., et al. (2026) *python-periodictable/periodictable: R2.1.0 2026-02-27..*

Meija, J., Coplen, T. B., Berglund, M., et al. (2016) *Atomic weights of the elements 2013 (IUPAC Technical Report)*. *Pure and Applied Chemistry***88**, 265.

A Code

```
class AccretionResult:
    def __init__(self, index, mixing_mass, final_mu) ->
        ↪ None:
            self.index = index
            self.mixing_mass = mixing_mass
            self.final_mu = final_mu
            self.mass_profile: None | NDArray[np.float64] =
                ↪ None
            self.mu_profile_mix: None | NDArray[np.float64] =
                ↪ None
            self.mu_profile_original: None |
                ↪ NDArray[np.float64] = None

    def add_integration_result(self, mass:
        ↪ NDArray[np.float64], mu_profile:
        ↪ NDArray[np.float64], mu_profile_original:
        ↪ NDArray[np.float64]) -> None:
            self.mass_profile = mass
            self.mu_profile_mix = mu_profile
            self.mu_profile_original = mu_profile_original

class AccretorProfile:
    def __init__(self, profile: mr.MesaData) -> None:
        self.center_h1 = float(profile.center_h1)
        self.age = float(profile.star_age)
        self.profile = profile
        self.mass =
            ↪ np.array(profile.mass, dtype=np.float64)
        self.mu = np.array(profile.mu, dtype=np.float64)
        arg = np.argmin(self.mu)
        self.mu[:arg] = self.mu[arg]

class AccretorProfiles:
    def __init__(self) -> None:
        self.profiles = self.__get_profiles()

    def __get_profiles(self, fresh = False) ->
        ↪ dict[float, list[AccretorProfile]]:
            if fresh:
                print("In Accretor.__get_profiles:\n\tloading
                    ↪ profiles")
                profiles_dict: dict[float,
                    ↪ list[AccretorProfile]] = {}
                for i, _ in enumerate(range(37)):
                    profiles: list[AccretorProfile] = []
                    mass = np.round(0.8+0.1*i, 1)
                    print(f"\tloading mass {mass}")
                    for j in range(1, 41):
                        profile = mr.MesaData(

                            ↪ f"/home/koen/master-internship/mesa-models/single-ms-stars/"+

                            ↪ f"M{mass}/LOGS/MS/profile{j}.data"
                        )

                        ↪ profiles.append(AccretorProfile(profile))
                    profiles_dict[mass] = profiles

                with open(

                    ↪ f"/home/koen/master-internship/data/accretor-cache/profiles.pkl",
                    "wb",
                ) as f:
                    pickle.dump(profiles_dict, f,
                        ↪ protocol=pickle.HIGHEST_PROTOCOL)

                return profiles_dict

            else:
                try:
                    with open(

                        ↪ f"/home/koen/master-internship/data/accretor-cache/profiles.pkl",
                        "rb",
                    ) as f:
                        return pickle.load(f)

                except FileNotFoundError:
                    return self.__get_profiles(fresh = True)
```

```

class Accretor:
    def __init__(self, profiles:AccretorProfiles,age:
        ↳ float, mass: float) -> None:
            self.mass = mass
            self.age = age

            self.profiles = profiles.profiles
            self.masses = self.__determine_profile_masses()
            self.central_h1 = self.__get_center_h1()
            self.mu_curve, self.mass_curve =
                ↳ self.__get_mu_mass_curve()

    def __get_center_h1(self) -> float:
        interp_ages: list[float] = []
        for mass in self.masses:
            mass_profiles: list[AccretorProfile] =
                ↳ self.profiles[mass]
            h1s: list[float] = []
            ages: list[float] = []
            for profile in mass_profiles:
                h1s.append(profile.center_h1)
                ages.append(profile.age)

            interp_ages.append(np.interp(self.age, ages,
                ↳ h1s))

        return np.interp(self.mass, self.masses,
            ↳ interp_ages)

    def __determine_profile_masses(self) -> list[float]:
        masses: list[float] = []

        profile_masses: list[float] =
            ↳ list(self.profiles.keys())

        x: float = self.mass
        i: np.intp = np.searchsorted(profile_masses, x)

        lower: None | float
        upper: None | float

        if i == 0:
            lower = None
            upper = profile_masses[0]
        elif i == len(profile_masses):
            lower = profile_masses[-1]
            upper = None
        else:
            lower = profile_masses[i - 1]
            upper = profile_masses[i]

        if lower != None:
            masses.append(lower)
        if upper != None:
            masses.append(upper)

        return masses

    def __get_mu_mass_curve(self) ->
        ↳ tuple[NDArray[np.float64], NDArray[np.float64]] :

        if len(self.masses) == 1:
            mu, mass =
                ↳ self.__get_mu_mass_curve_per_mass(self.masses[0])
            mu = mu[::-1]
            # arg = np.argmin(mu)
            # mu[arg] = mu[arg]
            return mu , mass[::-1]

        mu_lower, mass_lower =
            ↳ self.__get_mu_mass_curve_per_mass(self.masses[0])
        mu_upper, mass_upper =
            ↳ self.__get_mu_mass_curve_per_mass(self.masses[1])

        mass = np.unique(np.concatenate([mass_lower,
            ↳ mass_upper]))
        mu_lower = np.interp(mass, mass_lower, mu_lower)
        mu_upper = np.interp(mass, mass_upper, mu_upper)

        # Interpolation fraction in central H
        f = (self.mass - self.masses[0]) /
            ↳ (self.masses[1] - self.masses[0])

        # Interpolate evolutionary state
        mu = (1 - f) * mu_lower + f * mu_upper
        mu = mu[::-1]
        # arg = np.argmin(mu)
        # mu[arg] = mu[arg]
        return mu, mass[::-1]

    def __get_mu_mass_curve_per_mass(self, mass) ->
        ↳ tuple[NDArray[np.float64], NDArray[np.float64]] :
        surrounding_profiles: list[AccretorProfile] = []

        profiles = self.profiles[mass][::-1]
        all_center_h1s: list[float] = []
        for profile in profiles:
            all_center_h1s.append(profile.center_h1)

        x: float = self.central_h1
        i: np.intp = np.searchsorted(all_center_h1s, x)

        lower: None | AccretorProfile
        upper: None | AccretorProfile

        if i == 0:
            lower = None
            upper = profiles[0]
        elif i == len(all_center_h1s):
            lower = profiles[-1]
            upper = None
        else:
            lower = profiles[i - 1]
            upper = profiles[i]

        if lower != None:
            surrounding_profiles.append(lower)
        if upper != None:
            surrounding_profiles.append(upper)

        if len(surrounding_profiles) == 1:
            profile = surrounding_profiles[0]
            return np.array(profile.mu[::-1]),
                ↳ np.array(profile.mass[::-1])

        mass_lower =
            ↳ np.array(surrounding_profiles[0].mass)[::-1]
        mu_lower =
            ↳ np.array(surrounding_profiles[0].mu)[::-1]
        mass_upper =
            ↳ np.array(surrounding_profiles[1].mass)[::-1]
        mu_upper =
            ↳ np.array(surrounding_profiles[1].mu)[::-1]
        mass = np.unique(np.concatenate([mass_lower,
            ↳ mass_upper]))

        mu_lower = np.interp(mass, mass_lower, mu_lower)
        mu_upper = np.interp(mass, mass_upper, mu_upper)

        # Interpolation fraction in central H
        f = (self.central_h1 -
            ↳ surrounding_profiles[0].center_h1) /
            ↳ (surrounding_profiles[1].center_h1 -
            ↳ surrounding_profiles[0].center_h1)

        # Interpolate evolutionary state
        mu = (1 - f) * mu_lower + f * mu_upper

        return mu, mass

    def effective_mu_vs_depth(self, M_acc: float, mu_acc:
        ↳ float, save_profile: bool = False):
        ms = self.mass_curve

        ms = -np.diff(self.mass_curve)
        mus = (self.mu_curve[1:] + self.mu_curve[::-1]) /
            ↳ 2
        mu_current = mu_acc
        m_current = M_acc

        mu_profile = [mu_acc]

        crossing_index: int = 0
        for i, (m, mu) in enumerate(zip(ms, mus)):
            mu_last = mu_current
            mu_current = (m + m_current) / ((m / mu) +
                ↳ (m_current / mu_current))
            m_current = m + m_current
            mu_profile.append(mu_current)
            if mu < mu_current:
                crossing_index = np.min([i + 3,
                    ↳ len(mus)])
            ↳ len(mus)])
        mu_profile = np.array(mu_profile,
            ↳ dtype=np.float64)

        mu_start = np.array([mus[0], mus[0]],
            ↳ dtype=np.float64)
        mu_profile_original = np.concatenate([mu_start,
            ↳ mu_profile])
        mu_profile = np.concatenate([[mu_acc],
            ↳ mu_profile])
        mass = np.concatenate([[self.mass_curve[0] +
            ↳ M_acc], self.mass_curve])
        result = AccretionResult(crossing_index,
            ↳ self.mass_curve[0] -mass[crossing_index],
            ↳ mu_profile[crossing_index])

        if save_profile:
            result.add_integration_result(mass,
                ↳ mu_profile, mu_profile_original)

        return result

```